# Contents

**CI/CD — Deep Dive Notes** **1**
1. CI/CD Concepts . . . 1
   - Continuous Integration (CI) . . . 1
   - Continuous Delivery (CD) . . . 1
   - Continuous Deployment (CD) . . . 1
2. Pipeline Design Principles . . . 1
   - Shift-Left Testing . . . 1
   - Pipeline Stages . . . 2
   - The Four Key Metrics (DORA) . . . 2
3. Jenkins . . . 2
   - Declarative Pipeline . . . 2
   - Jenkins Shared Libraries . . . 4
4. GitHub Actions . . . 4
   - Reusable Workflow . . . 4
   - Matrix Builds + Caching . . . 5
5. GitLab CI . . . 6
6. ArgoCD — GitOps . . . 7
   - Install & Setup . . . 7
   - Application Definition . . . 7
   - App of Apps Pattern . . . 8
7. Deployment Strategies . . . 8
   - Blue-Green . . . 8
   - Canary . . . 8
   - Rolling Update (Kubernetes default) . . . 8
   - Feature Flags . . . 8

 Resources . . . 8

# CI/CD — Deep Dive Notes

## 1. CI/CD Concepts

### Continuous Integration (CI)

- Every commit triggers automated build + test
- Fast feedback loop (< 10 min target)
- Keeps main branch always buildable
- Catches integration bugs early

### Continuous Delivery (CD)

- Automated release to production-like environment
- Human approval gate before production deploy
- Every commit is potentially releasable

### Continuous Deployment (CD)

- Every passing build auto-deploys to production
- No human gate
- Requires high test coverage + observability + rollback capability

---

## 2. Pipeline Design Principles

### Shift-Left Testing

```
Unit Tests → Integration Tests → E2E Tests → Performance → Security → Deploy
  (fast)        (medium)          (slow)     (scheduled)   (gate)
```

### Pipeline Stages

```
1. Source       → checkout, validate
2. Build        → compile, lint, SAST scan
3. Unit Test    → fast isolated tests
4. Docker Build → multi-stage, cache layers
5. Integration  → spin up dependencies, test
6. Security     → container scan, DAST, secrets scan
7. Staging      → deploy to staging, smoke test
8. E2E          → full flow tests
9. Production   → deploy with approval gate
10. Post-deploy → health check, notify
```

### The Four Key Metrics (DORA)

| Metric | Elite | High | Medium | Low |
|---|---|---|---|---|
| Deployment Frequency | On demand (multiple/day) | Weekly | Monthly | <6 months |
| Lead Time for Changes | <1 hour | 1day-1week | 1week-1month | >6 months |
| Change Failure Rate | 0-15% | 16-30% | 16-30% | 46-60% |
| MTTR | <1 hour | <1 day | <1 week | >6 months |

---

## 3. Jenkins

### Declarative Pipeline

```
pipeline {
    agent {
        kubernetes {
            yaml '''
                apiVersion: v1
                kind: Pod
                spec:
                  containers:
                  - name: maven
                    image: maven:3.9-eclipse-temurin-17
                    command: [cat]
                    tty: true
                  - name: docker
                    image: docker:24-dind
                    securityContext:
                      privileged: true
            '''
        }
    }

    options {
        timeout(time: 30, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '10'))
        disableConcurrentBuilds()
    }

    environment {
        REGISTRY = 'registry.company.com'
        IMAGE = "${REGISTRY}/myapp:${GIT_COMMIT.take(8)}"
        SONAR_TOKEN = credentials('sonar-token')
```

```
    }

    stages {
        stage('Build & Test') {
            steps {
                container('maven') {
                    sh 'mvn clean verify -T 4 --no-transfer-progress'
                    junit 'target/surefire-reports/**/*.xml'
                    publishHTML([
                        allowMissing: false,
                        reportDir: 'target/site/jacoco',
                        reportFiles: 'index.html',
                        reportName: 'Coverage Report'
                    ])
                }
            }
        }

        stage('SonarQube') {
            steps {
                container('maven') {
                    withSonarQubeEnv('sonarqube') {
                        sh 'mvn sonar:sonar'
                    }
                    timeout(time: 5, unit: 'MINUTES') {
                        waitForQualityGate abortPipeline: true
                    }
                }
            }
        }

        stage('Docker Build & Push') {
            steps {
                container('docker') {
                    sh '''
                        docker build --build-arg VERSION=${GIT_COMMIT} \
                          --cache-from ${IMAGE}:latest \
                          -t ${IMAGE} -t ${REGISTRY}/myapp:latest .
                        docker push ${IMAGE}
                        docker push ${REGISTRY}/myapp:latest
                    '''
                }
            }
        }

        stage('Deploy Staging') {
            when { branch 'main' }
            steps {
                sh "helm upgrade --install myapp ./helm --set image.tag=${GIT_COMMIT.take(8)} -n
                sh './scripts/smoke-test.sh staging'
            }
        }

        stage('Deploy Production') {
            when { branch 'main' }
            input {
                message "Deploy to production?"
                ok "Deploy"
                submitter "tech-leads"
            }
```

```
            steps {
                sh "helm upgrade --install myapp ./helm --set image.tag=${GIT_COMMIT.take(8)} -n
                sh './scripts/smoke-test.sh production'
            }
        }
    }

    post {
        always {
            cleanWs()
        }
        failure {
            slackSend channel: '#alerts', color: 'danger',
              message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER} - ${env.BUILD_URL}"
        }
        success {
            slackSend channel: '#deployments', color: 'good',
              message: "DEPLOYED: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
        }
    }
}
```

**Jenkins Shared Libraries**

```
// vars/dockerBuild.groovy
def call(Map config) {
    def image = "${config.registry}/${config.name}:${env.GIT_COMMIT.take(8)}"
    docker.build(image, "--build-arg VERSION=${env.GIT_COMMIT} .")
    docker.withRegistry("https://${config.registry}", config.credentials) {
        docker.image(image).push()
    }
    return image
}

// Jenkinsfile usage:
@Library('company-shared-lib') _
def img = dockerBuild(registry: 'registry.company.com', name: 'myapp', credentials: 'registry-cre
```

---

## 4. GitHub Actions

**Reusable Workflow**

```
# .github/workflows/reusable-docker.yml
on:
  workflow_call:
    inputs:
      image-name:
        required: true
        type: string
    outputs:
      image-tag:
        value: ${{ jobs.build.outputs.tag }}
    secrets:
      registry-password:
        required: true
```

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      tag: ${{ steps.meta.outputs.version }}
    steps:
      - uses: actions/checkout@v4
      - uses: docker/metadata-action@v5
        id: meta
        with:
          images: ${{ inputs.image-name }}
          tags: |
            type=sha
            type=semver,pattern={{version}}
            type=edge,branch=main
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

# Calling workflow:
jobs:
  build:
    uses: ./.github/workflows/reusable-docker.yml
    with:
      image-name: ghcr.io/myorg/myapp
    secrets:
      registry-password: ${{ secrets.GHCR_TOKEN }}
```

**Matrix Builds + Caching**

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python: ['3.10', '3.11', '3.12']
        os: [ubuntu-latest, windows-latest]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
          cache: 'pip'
      - uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements*.txt') }}
          restore-keys: |
            ${{ runner.os }}-pip-
      - run: pip install -r requirements.txt && pytest
```

---

### 5. GitLab CI

```yaml
# .gitlab-ci.yml
include:
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/Container-Scanning.gitlab-ci.yml

variables:
  DOCKER_DRIVER: overlay2
  DOCKER_TLS_CERTDIR: '/certs'

stages: [build, test, scan, deploy-staging, deploy-prod]

.common: &common
  image: docker:24
  services: [docker:24-dind]
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY

build:
  <<: *common
  stage: build
  script:
    - docker pull $CI_REGISTRY_IMAGE:latest || true
    - docker build --cache-from $CI_REGISTRY_IMAGE:latest
        -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
        -t $CI_REGISTRY_IMAGE:latest .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
    - docker push $CI_REGISTRY_IMAGE:latest
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

unit-tests:
  stage: test
  image: python:3.11
  script:
    - pip install -r requirements.txt
    - pytest --junitxml=report.xml --cov=src --cov-report=xml
  coverage: '/TOTAL.*\s+(\d+%)$/'
  artifacts:
    reports:
      junit: report.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml

deploy-staging:
  stage: deploy-staging
  image: alpine/helm:3
  environment:
    name: staging
    url: https://staging.example.com
  script:
    - helm upgrade --install myapp ./helm
        --set image.tag=$CI_COMMIT_SHA
        -n staging --wait
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

```yaml
deploy-production:
  stage: deploy-prod
  image: alpine/helm:3
  environment:
    name: production
    url: https://example.com
  script:
    - helm upgrade --install myapp ./helm
        --set image.tag=$CI_COMMIT_SHA
        -n production --wait
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      when: manual
```

---

## 6. ArgoCD — GitOps

### Install & Setup

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/in
kubectl port-forward svc/argocd-server -n argocd 8080:443
argocd login localhost:8080
```

### Application Definition

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: myapp
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/myorg/gitops-repo
    targetRevision: main
    path: apps/myapp/overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
      prune: true        # delete resources not in git
      selfHeal: true     # auto-sync on drift
    syncOptions:
      - CreateNamespace=true
      - ApplyOutOfSyncOnly=true
    retry:
      limit: 3
      backoff:
        duration: 5s
        factor: 2
        maxDuration: 3m
```

### App of Apps Pattern

```yaml
# apps/root-app.yaml — manages all other apps
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root-app
spec:
  source:
    path: apps/            # directory of Application manifests
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

---

## 7. Deployment Strategies

### Blue-Green

```
Blue (live) → Green (new version deployed, tested)
→ Switch LB to Green → Blue becomes standby
Rollback: switch back to Blue instantly
```

### Canary

```
90% traffic → v1 (stable)
10% traffic → v2 (canary)
Monitor metrics → gradually shift if healthy
```

### Rolling Update (Kubernetes default)

```yaml
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0        # never reduce capacity
    maxSurge: 1              # allow one extra pod during update
```

### Feature Flags

```python
# Using LaunchDarkly / Unleash
if feature_flags.is_enabled("new-checkout-flow", user):
    return new_checkout()
else:
    return old_checkout()
```

## Resources

- https://www.jenkins.io/doc/book/pipeline/ — Jenkins Pipeline docs
- https://docs.github.com/en/actions — GitHub Actions docs
- https://argo-cd.readthedocs.io/ — ArgoCD docs
- https://codefresh.io/learn/ci-cd/ — CI/CD concepts
- https://dora.dev/ — DORA metrics research